

Atividade em Sala — Manipulando Dados com Ponteiros - Individual

Nome: _____

RM: _____

LEIA COM ATENÇÃO ATÉ O FINAL.

Objetivo

Compreender na prática como um ponteiro armazena o endereço de outra variável e como ele pode ser utilizado para consultar e modificar valores diretamente na memória

Desafio

Desenvolva um programa em C que:

1. Declare uma variável inteira chamada "valor".
 2. Declare um ponteiro "ptr" para essa variável.
 3. Armazene em "ptr" o endereço de "valor".
 4. Exiba:
 - o valor original;
 - o endereço da variável;
 - o endereço armazenado no ponteiro;
 - o valor acessado por meio do ponteiro.
 5. Altere o conteúdo de "valor" somente utilizando o ponteiro.
 6. Exiba novamente o valor da variável para comprovar que ela foi modificada.
-

Código inicial

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int valor = 10;
```

```
    int *ptr = NULL;
```

```
    // 1. Faça ptr apontar para valor
```

```
    // 2. Exiba o valor de 'valor'
```

```
// 3. Exiba o endereço de 'valor'

// 4. Exiba o endereço armazenado em ptr

// 5. Exiba o valor utilizando *ptr

// 6. Altere valor para 50 utilizando somente ptr

// 7. Exiba novamente valor

return 0;

}
```

Resultado esperado

A saída deve apresentar algo semelhante a:

Valor original: 10

Endereço de valor: 0x7ffd...

Endereço armazenado em ptr: 0x7ffd...

Valor acessado por ptr: 10

Valor depois da alteração: 50

Os endereços de memória serão diferentes em cada computador.

Parte 2 — Ponteiro como parâmetro de uma função

Agora crie uma função:

```
void dobrar(int *numero);
```

A função deverá receber o endereço de uma variável e dobrar seu conteúdo.

Exemplo:

```
int x = 10;
```

```
dobrar(&x);
```

```
printf("%d\n", x);
```

Resultado esperado:

20

Implemente a função sem utilizar "return" para devolver o novo valor.

Parte 3 — Questões para análise

Responda no próprio código utilizando comentários:

1. O que significa o operador "&" em:

```
ptr = &valor;
```

2. O que significa o operador "*" em:

```
printf("%d", *ptr);
```

3. Qual é a diferença entre:

ptr

e

*ptr

4. Por que esta instrução altera a variável original?

```
*ptr = 50;
```

5. Por que utilizamos:

```
dobrar(&x);
```

e não simplesmente:

```
dobrar(x);
```

6. O que poderia acontecer se utilizássemos um ponteiro sem inicializá-lo?

Desafio extra — Ponteiro + Struct

Considere:

```
typedef struct {  
  
    int rm;  
  
    float nota;  
  
} Aluno;
```

Crie:

```
Aluno aluno = {1234, 7.0};
```

```
Aluno *ptrAluno = &aluno;
```

Altere a nota para "9.5" utilizando obrigatoriamente:

```
ptrAluno->nota
```

Depois reescreva a mesma operação utilizando:

```
(*ptrAluno).nota
```

Explique por que as duas formas são equivalentes.

Conceitos que devem aparecer na atividade

Ao final, você deve conseguir explicar:

&variavel → endereço da variável

ponteiro → armazena um endereço

*ponteiro → conteúdo existente nesse endereço

ptr->campo → campo de uma struct acessada por ponteiro

Regra de segurança

Antes de acessar o conteúdo de um ponteiro, verifique se ele possui um endereço válido:

```
if (ptr != NULL) {  
  
    printf("%d\n", *ptr);  
  
}
```

Entregar até o final da aula:

- código ".c" funcionando;

- respostas das seis questões em comentários;
- print da execução do programa;
- implementação da função "dobrar";
- desafio extra utilizando "struct" e operador "->".